

LayerSkip: Per-Layer Analysis on LayerSkip and Base Llama Models

Mohamed Zahran Marcin Wizgird Eric Sung Hassaan Rafique
Georgia Institute of Technology
Atlanta, GA

mzahran6@gatech.edu, marcin.wizgird@gatech.edu, msung33@gatech.edu, hrafique3@gatech.edu

Abstract

LayerSkip models enhance Transformer efficiency through layer dropout during training and adaptive computation during inference. Understanding the internal mechanisms that enable this efficiency requires a detailed comparison with baseline architectures. In this study, we compare the LLaMA-3.2-1B model to its LayerSkip counterpart using mechanistic interpretability techniques, focusing on residual stream dynamics and layer activations. We analyze the layer-wise evolution of prediction confidence through log-probability contributions, identify the effective prediction depth of each model, and evaluate representational similarity using Centered Kernel Alignment (CKA) on residual stream outputs. To complement these analyses, we conduct experiments involving layer ablation, model probing with intermediate activations on downstream tasks, and performance evaluation under simulated early exit. We find that LayerSkip exhibits faster confidence buildup from early layers, effectively predicts the final token significantly earlier in mid-layers and has layers more uniformly contributing to the final prediction. Moreover, the higher layers learn more rich representations shown by the Center Kernel Alignment analysis, which is further confirmed when fine-tuning the model using domain-specific dataset.

1. Introduction/Background/Motivation

Model explainability is critical as neural network architectures scale. LayerSkip models [3] aim to accelerate Transformers’ inference by using layer dropout during training and apply early exiting and speculative decoding during inference, achieving efficiency while maintaining quality. Although their effectiveness has been proven empirically, the internal mechanism of the models has not been fully understood.

Our primary research goal is to mechanistically compare LayerSkip with its base Llama counterpart, identifying how information is processed through each layer and how feature representations evolve within the residual stream. We hy-

pothesize that LayerSkip’s specialized training method induces functional differences throughout the layers, allowing for effective computation even with reduced layer depth or skipped layers. For investigation, we employ several analysis techniques inspired by Mechanistic Interpretability [7]:

1. **Log Probability Contribution Analysis:** Examining how the log probability of the final predicted token evolves based on the residual stream state after each sub-layer (attention, MLP), providing a “Logit Lens” view [5].
2. **Effective Prediction Layer Analysis:** Deriving from the log probability analysis, we identify the average layer depth at which each model reaches stable and confident predictions.
3. **Representational Similarity (CKA):** Using Centered Kernel Alignment [4] to measure the similarity between residual stream representations produced after each full transformer layer.
4. **Performance With Simulated Early Exit:** Evaluating model performance (accuracy, loss) under simulated early exit scenarios.
5. **Layer Ablation:** Assessing the importance of individual layers by measuring performance degradation when a layer is ablated.
6. **Model Probing:** Evaluating the quality of intermediate representations by using them as input for a downstream sentiment classification task.

By combining these perspectives, particularly focusing on the residual stream dynamics (1, 2, 3), we aim to uncover the mechanistic differences that enable LayerSkip’s efficiency. Understanding these differences can inform efficient model designs for future implementations.

Our primary research goal is to analyze how activations of LayerSkip models differ from those of the base Llama model, and how both models react to layer skipping and early exiting during inference. Specifically, we aim to examine the evolution and distribution of activations and output logits across layers during inference. By adopting established interpretability methodologies, we prioritize experi-

mental setup with special focus on LayerSkip’s early exit capabilities.

This research is relevant for developing more computationally efficient Large Language Models (LLMs) and providing insights into how specific training strategies, such as layer dropout alter the internal function of transformer layers. Successful identification of these mechanisms could lead to more targeted approaches for model compression and faster inference. Understanding these patterns may inform the development of alternative algorithms or adaptations to other domains, offering broader applicability beyond the original architecture.

2. Approach

Our analysis compares the Llama-3.2-1B and LayerSkip Llama-3.2-1B models. Llama 3.2-1B is a multilingual, text-only lightweight LLM with ability to perform multiple natural language processing tasks, such as translation, code generation, and reasoning. Due to hardware constraints, we focus on these 1B parameter variants.

Experiments are conducted using PyTorch and the ‘TransformerLens’ library [6] for mechanistic interpretability and access to internal activations via hooks. Our primary focus is on the residual stream states, including the post-embedding, post-attention and post-MLP outputs. These states form the basis for our log probability, CKA analyses and other experiments.

2.1. Finetuning Approach

We utilized Google Colab Pro + with an NVIDIA A100 40GB GPU to conduct fine-tuning experiments. We opted for the Guanaco-2-10k dataset, a small-scaled multi-lingual prompt dataset designed for linguistic and grammatical tasks. Given its relatively small size compared to other larger datasets, such as WikiText-103, this dataset provided functional advantage for our resource constraints.

To optimize the fine-tuning process, we used the transformer training library, Optuna. The best hyperparameters identified for the LLaMA 3.2-1B model for the Guanaco-2-1k dataset are included in the Appendix 5.2.

In the original LayerSkip paper, the model training was performed with dynamic layer dropout applied throughout the training process. After reviewing alternative training methods, we decided to utilize the LayerSkipSFTTrainer module available in the TLR Library from Hugging Face [2], which enables dynamic layer skipping during fine-tuning of a pre-trained model rather than requiring training from scratch. This approach is ideal for us considering our limited resources and the sheer size of the LLMs.

We conducted experiments across four configurations: (1) QLoRA with LayerSkip fine-tuning, (2) QLoRA with standard fine-tuning (no LayerSkip), (3) standard fine-

tuning with LayerSkip (no QLoRA), and (4) standard fine-tuning without QLoRA or LayerSkip.

2.2. Log Probability Contribution Analysis

To understand how prediction confidence evolves, we adopt a Logit Lens approach [5]. For a given input prompt and its corresponding final model prediction (let the predicted token index be t_{pred}), we run the model and cache intermediate residual stream states s_k at the final token position after each transformer block sub-layer discussed above (attention and MLP sub-layers before normalization).

For each cached state s_k , we simulate the final output projection by applying the model’s final layer normalization operation (LN_{final} , corresponding to the `ln.final` layer) followed by multiplication with the unembedding matrix (W_U , corresponding to the `unembed` layer weights) to obtain intermediate logits l_k :

$$l_k = W_U \cdot LN_{\text{final}}(s_k) \quad (1)$$

We then compute the log probability assigned to the actual final predicted token t_{pred} based on these intermediate logits using the LogSoftmax function:

$$\text{logprob}_k(t_{\text{pred}}) = \text{LogSoftmax}(l_k)_{t_{\text{pred}}} \quad (2)$$

This process yields a trajectory of log probabilities $\{\text{logprob}_k(t_{\text{pred}})\}$ across the components k , showing how the model’s confidence in the final answer builds up through its layers and sublayers. We compute this trajectory for multiple prompts from the ARC dataset and average the results for comparison between the base Llama and LayerSkip models.

2.3. Effective Prediction Layer Analysis

Using the average log probability trajectories generated above 2.2, we analyze at which layer depth the model consistently reaches a high confidence level for the final predicted token. We define a threshold (e.g., 90% of the maximum average log probability achieved by the full model) and determine the first layer or sub-layer index where the average log probability stably crosses this threshold. This provides a quantitative measure of how early each model effectively ”settles” on its final prediction.

2.4. Representational Similarity (CKA) Analysis

We measure the similarity between representations produced by different layers using linear Centered Kernel Alignment (CKA) [4]. For a batch of m prompts from the ARC dataset [1], we extract the activation vectors at the final token position from the output of each full transformer layer i (using the `blocks.{i}.hook.resid.post` hook). This yields activation matrices $X_i \in \mathbb{R}^{m \times d}$, where

m is the number of prompts (samples) and d is the model’s hidden dimension (features).

We compute the linear CKA score between the activations X_i and X_j for all pairs of layers (i, j) . This involves the following steps:

1. Calculate the Gram matrices $K_i, K_j \in \mathbb{R}^{m \times m}$, which measure similarity between samples:

$$K_i = X_i X_i^T, \quad K_j = X_j X_j^T \quad (3)$$

2. Define the $m \times m$ centering matrix H :

$$H = I_m - \frac{1}{m} \mathbf{1} \mathbf{1}^T \quad (4)$$

where I_m is the identity matrix and $\mathbf{1}$ is an $m \times 1$ column vector of ones.

3. Center the Gram matrices using H :

$$K_{ic} = H K_i H, \quad K_{jc} = H K_j H \quad (5)$$

(Note: In practice, this is computed efficiently without explicitly forming H).

4. Compute the CKA score using the Hilbert-Schmidt Independence Criterion (HSIC) formulation based on the trace (Tr) of matrix products:

$$\text{CKA}(X_i, X_j) = \frac{\text{Tr}(K_{ic} K_{jc})}{\sqrt{\text{Tr}(K_{ic}^2) \text{Tr}(K_{jc}^2)}} \quad (6)$$

(A small epsilon is added to the denominator in implementation for numerical stability).

We compute this score for all layer pairs (i, j) to form a CKA similarity matrix. Finally, we average these matrices across multiple batches of prompts from the ARC dataset.

2.5. Simulated Early Exiting

We evaluate the impact of early exiting on LayerSkip and Llama by truncating all subsequent layers after a given layer depth. This is done on the base modes, as well as a fine-tuned version of the LayerSkip model, trained on the specific dataset, which is assessed to examine the effects of domain-specific adaptation.

Token generation experiments are conducted on the Guanaco-2-1k dataset using stepwise autoregressive evaluation. For each sub-sequence of the dataset text, the models with reduced layers predict the next token, which is then compared against the ground truth to compute the token accuracy. We also evaluate cross-entropy loss to monitor context generalization and model robustness toward layer alteration.

2.6. Layer Ablation Studies

These experiments attempted to view the impact of ablating different layers of the model. This would show the relative impact of each layer on the loss when predicting the next output token.

The experiment was set up using a movie synopsis dataset. The examples were selected and truncated at 250,500,1000 and 2000 words, allowing analysis at different input sequence lengths. For each example, the model is run once while ablating each full layer to 0, with the resulting ablated loss being compared to the original model loss to find the relative degradation in performance. This experiment is performed for both LayerSkip and the Llama model and attempts to compare the relative importance of each model layer to the overall output performance.

2.7. Model Probing And Activation Analysis

In this section we attempt to analyze the general differences between the activations in LayerSkip and Llama. We use a different dataset in this experiment, a set of tweets with their sentiment classification into negative, neutral, and positive. We attempt to determine the quality of extracted features by using the intermediate layer outputs of the residual connections of each layer as inputs to a simple one layer classifier that attempts to learn the labels of the input examples. Each layer’s activations are kept independent from the others and are used to train a model separately, with the training performances of each model being recorded and analyzed to determine how successful the model is at classifying input labels given these different inputs. All model hyperparameters are kept constant between the two experiments to ensure a fair comparison of the input features.

2.8. Challenges

One limitation we encountered was the ability to study the LayerSkip model with live early exit and speculative decoding. This is due to the fact that the Mechanistic Interpretability libraries used did not support these models, so instead we loaded standard Llama model architectures while modifying the weights to match those of LayerSkip. Although we were unable to utilize speculative decoding in our study, we observed that the layer-by-layer interpretation was sufficient to produce meaningful results in the analysis of activation patterns and logit outputs at different layers, simulating early exit, and achieving insights into how the model is different from the base Llama version.

3. Experiments and Results

Our experiments aim to identify distinctive mechanistic differences between LayerSkip and base Llama models that explain LayerSkip’s efficiency. We analyze residual stream dynamics using log probability evolution and

Model Configuration	Eval Loss	Perplexity
LLaMA (baseline, no QLoRA)	2.5470	12.77
LLaMA + QLoRA	1.5691	4.80
LLaMA + LayerSkip (no QLoRA)	2.9121	18.40
LLaMA + QLoRA + LayerSkip (ours)	1.9758	7.21

Table 1. Fine-tuned Model Evaluation Results

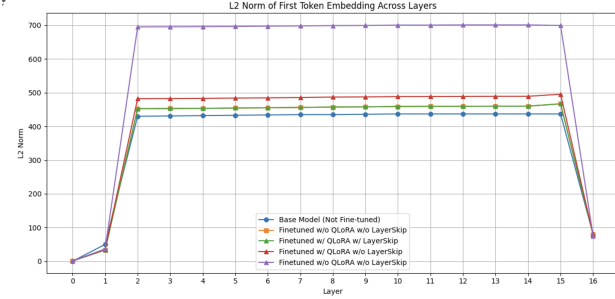


Figure 1. Embedding Norm at each Layer during Fine-tuning

representational similarity (CKA), complemented by functional evaluations performed by the team (performance under layer removal, layer ablation, and model probing). We used the Llama-3.2-1B architecture for both the base and LayerSkip models. Log probability and CKA analyses utilized prompts from the ARC dataset [1].

3.1. Finetuning

QLoRA greatly boosts performance by cutting perplexity from 12.77 to 4.80, while LayerSkip on its own performs worse than the baseline—yet when combined, they strike a good balance between efficiency and performance, improving over LayerSkip alone though not quite matching QLoRA’s best result.

Embedding Norms at Each Layer:

From figure 1 we can see that the Base model has low embedding norms across layers. After fine-tuning, embedding norms increase as expected. The finetuned layer-skip model shows lower embedding norms than the non-layer-skip model, indicating better regularization and reduced overfitting. While Finetuned without layerskip and without Qlora experiences the greatest norm which suggests that this model has the highest chance of overfitting.

3.2. Log Probability Contribution (Logit Lens)

We first examined how the model’s confidence in its final prediction evolves across layers using the Logit Lens approach described in Section 2.2. Figure 2 plots the average log probability assigned to the final predicted token, calculated from the residual stream state after each sub-layer (Attention and MLP), averaged over tokens generated on ARC-Easy prompts.

The trajectories reveal a stark difference. The base Llama model (blue line) shows a gradual increase in log

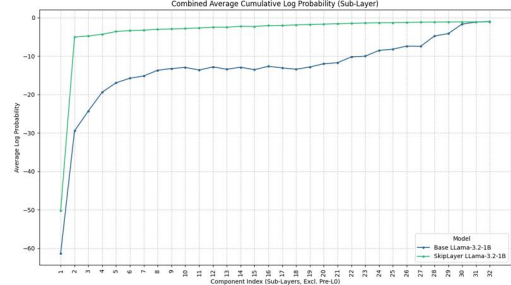


Figure 2. Average Log Probability Progression for Final Predicted Token (Base Llama vs. LayerSkip) after each sub-layer (excluding Pre-L0). Based on ARC-Easy dataset.

probability, indicating that prediction confidence builds relatively slowly and relies significantly on processing through deeper layers. In contrast, the LayerSkip model (green line) exhibits a much faster rise, achieving high confidence within the first few layers and maintaining it (plateauing) through the middle layers. This suggests that LayerSkip consolidates the information necessary for prediction much earlier in the network compared to its base counterpart.

3.3. Effective Prediction Layer

Building on the log probability analysis, we quantified *when* each model effectively finalizes its prediction. Analyzing 200 token predictions (20 prompts $\times$ 10 generated tokens) on ARC-Easy, we determined the layer after which the log probability of the predicted token stabilized above a confidence threshold (relative to the final layer’s confidence).

The results show a significant difference in the prediction timing (Figure 3). For the **base Llama model**, negligible prediction stabilization occurs before layer 11 (3.0%). Confidence builds rapidly only in the final layers, with 35.0% of tokens effectively predicted after Layer 15 and another 39.5% after Layer 16 (totaling 74.5% in the last two layers). This indicates a strong reliance on the full network depth.

Conversely, the **LayerSkip model** demonstrates much earlier prediction stabilization. A notable 15.5% of tokens are effectively predicted after Layer 0, and 12.5% after Layer 2. Cumulatively, LayerSkip reaches over 50% effective prediction by Layer 9 and 10, and over 70% by Layer 12. This distributed and significantly earlier stabilization strongly supports the mechanistic basis for LayerSkip’s early exit capability – the necessary computation for confident prediction is often completed well before the final layers.

3.4. Representational Similarity via CKA

We used linear CKA to compare the similarity of the representations output by each full transformer layer (state af-

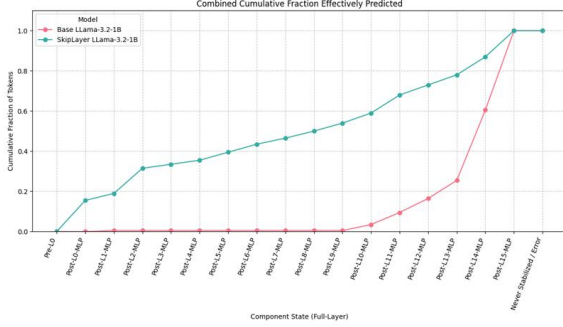


Figure 3. Cumulative percentage of effectively predicted tokens before particular layer

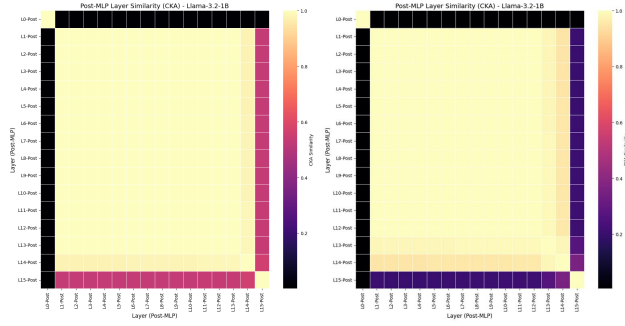


Figure 4. Average CKA Similarity Matrix between Layer Residual Outputs (hook_resid_post) on ARC dataset for Base Llama (left) and LayerSkip (right).

ter MLP, blocks. $\{i\}$.hook_resid_post) across 100 ARC prompts. Figure 4 heatmaps presents side-by-side the CKA similarity matrix heatmaps for both models.

Both models exhibit a clear block structure, where adjacent layers, especially in the middle of the network, produce highly similar representations (CKA > 0.9). This suggests functional grouping within the models. However, the LayerSkip model displays slightly higher similarity (CKA often ≈ 1.000) across a wider span of middle layers (approx. L1-Post to L13-Post) compared to the base Llama model (CKA ≈ 0.996 - 0.999 in the same range). This heightened similarity in its intermediate layers in both models suggests significant representational stability and redundancy. The higher mid-layer similarity in LayerSkip provides a potential mechanism for its greater robustness: if intermediate representations are highly similar, skipping a layer may have less impact on the information propagated forward. Both models show lower similarity between early/embedding layers and the late layer. The final layer’s (L15-Post) representation appears relatively distinct from preceding layers in both models, though the drop-off pattern might differ significantly. LayerSkip model derives significantly more diversified and novel features in final layers.

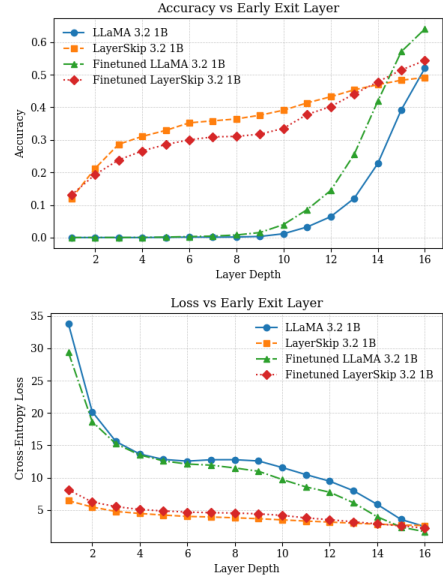


Figure 5. Token Accuracy and Cross Entropy Loss of Llama, LayerSkip, and Fine-Tuned Models When Varying the Exit Layer

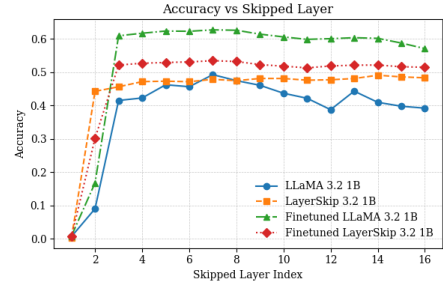


Figure 6. Token Accuracy of Llama, LayerSkip, and Fine-Tuned Models on Single Layer Skip at Varying Positions

3.5. Performance under Simulated Early Exit

Functional evaluations under simulated early exit align well with our mechanistic findings. The results in (Figure 5) mirror the results of the effective prediction layer (§3.3): LayerSkip achieves reasonable accuracy with very few layers active, while base Llama requires nearly the full model depth. This functionally validates LayerSkip’s ability to compute effectively at intermediate stages, as suggested by the faster confidence build-up (§3.2). Similarly, LayerSkip’s robustness to single layer skipping (Figure 6), especially in mid-layers, corresponds strongly with the high representational similarity observed via CKA (§3.4) in those same layers. The base Llama’s higher sensitivity to skips reflects its more distinct layer representations and later confidence consolidation. The crucial role of Layer 0 observed in both models during skips might indicate essential initial processing, consistent with LayerSkip still showing significant effective prediction gains after Layer 0 (§3.3).

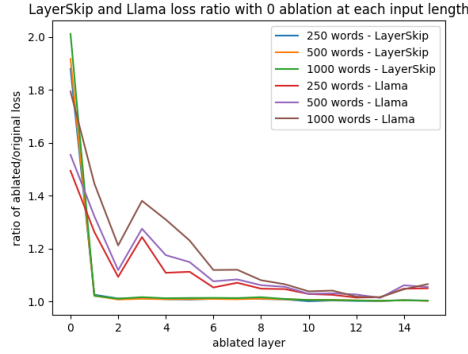


Figure 7. Next token cross entropy loss ratio for different sequence lengths with Llama and LayerSkip

3.6. Layer Ablation Results

Layer ablation studies (Figure 7) provide further evidence. The significantly lower increase in ablation to original loss ratio when ablating LayerSkip’s intermediate layers compared to base Llama corroborates the idea of functional redundancy suggested by the CKA analysis (§3.4). If representations are highly similar, removing one layer’s contribution has less impact. LayerSkip’s consistent ablation profile across different sequence lengths also suggests a more stable functional role for its layers compared to base Llama.

Segmenting input examples by sequence length also shows an interesting conclusion. Figure 7 shows that, while the Llama model showed clear variation in ablated losses the longer the sequence, this was not the case for LayerSkip, where the output confidence seemed robust regardless of the input sequence length.

3.7. Model Probing Results

Finally, model probing (Figure 8) adds another dimension. LayerSkip maintains high downstream task (sentiment classification) performance when probed using representations from most layers, whereas base Llama shows a performance dip for mid-to-late layer representations. This suggests that while LayerSkip’s mid-layer representations are highly similar (CKA analysis, §3.4), they effectively preserve linearly decodable task-relevant features. Base Llama’s representations, while potentially evolving more distinctly between layers, might obscure this information in intermediate stages before final processing. This finding reinforces the utility of LayerSkip’s intermediate representations for potential early exits without sacrificing usability for downstream tasks.

4. Conclusion

The following conclusions and recommendations stem from our experiments: **First, LayerSkip consolidates in-**

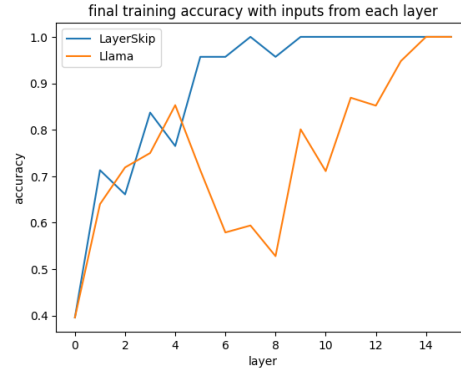


Figure 8. Final training accuracy for Llama and LayerSkip given activations as inputs to a simple classifier

formation early during inference. LayerSkip models achieve high prediction confidence significantly earlier in the model depth compared to the base model, which relies more heavily on later layers, supporting the viability of LayerSkip’s early-exit mechanism. **Second, LayerSkip model has a higher mid-layer redundancy.** CKA results reveal extremely high representational similarity (near 1.0) across many middle layers (approx. L1-L13 Post-MLP) of LayerSkip, suggesting functional redundancy. This likely underpins its observed robustness to layer skipping and ablation in these intermediate stages. **Third, speculative decoding depths can be optimized.** The findings strongly support using LayerSkip’s earlier layers (up to Layer 11-12) as an efficient and effective assistant model for speculative decoding due to early confidence and stable representations. The verifier model shall have over 16 layers as LayerSkip learns significantly new representations from that layers. **Moreover, further analysis is needed to understand the specific role of first layer of LayerSkip to its performance.** Lastly, **the LayerSkip analysis shall be extended to the attention-head level** so as to potentially enable more **targeted head-skipping regularization techniques** decreasing their activation sparsity.

References

- [1] allenai/ai2_arc · datasets at hugging face. 2, 4
- [2] huggingface/trl: Train transformer language models with reinforcement learning. 2
- [3] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12622–12642. 1
- [4] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations

Student Name	Contributed Aspects
Mohamed Zahran	Ablation studies and model probing
Marcin Wizgird	CKA, log contribution and effective token prediction analysis
Eric Sung	Simulated early exit and zero-shot classification experiments
Hassaan Rafique	Finetuning base Llama and LayerSkip models

Table 2. Contributions of team members.

revisited. 1, 2

- [5] nostalgebraist. interpreting GPT: the logit lens. 1, 2
- [6] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. 5(3):e00024.001. 2
- [7] Zeping Yu, Kailai Yang, Zhiwei Liu, and Sophia Ananiadou. Exploring the residual stream of transformers. version: 1. 1

5. Appendix

5.1. Experiment Source Code Repository

The following GitHub link directs to the shared repository that contains the scripts and notebooks used to perform experiments. [Code Repository](#)

5.2. Best Fine-tuning Hyperparameters

The best hyperparameters identified for the Llama 3.2-1B model for the Guanaco-2-1k dataset were the following:

- Learning Rate: 9.04×10^{-5}
- Weight Decay: 0.1888
- LoRA Dropout: 0.05
- LoRA Rank (r): 8
- LoRA Scaling Factor (α): 16

Due to memory constraints, we decided to use batch-size=1, as larger batch sizes require larger space in memory, which was not feasible within the available hardware limits.

5.3. Single-Layer Skipping Analysis

In addition to the simulated early exit experiments described in Section 2.5, we conducted a complementary analysis in which a single layer was explicitly removed at each position in the model. This approach emulates the effect of skipping individual layers during next-token prediction. By selectively omitting one layer at a time, we measured the resulting changes in token-level accuracy and cross-entropy loss. The results of this analysis are presented in Figure 9.

5.4. Zero-shot Classification Results

Similar to model probing in Section 2.7, we conducted an exploratory experiment involving zero-shot classification on a sentiment analysis task using the base LLaMA and

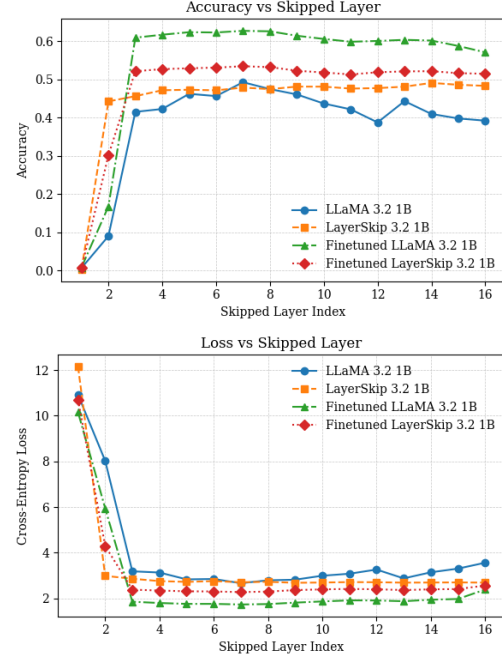


Figure 9. Token Accuracy and Cross Entropy Loss of Llama, LayerSkip, and Fine-Tuned Models with Layer Removal on the Layer Index

Class	Precision	Recall	F1-Score	Support
Negative	0.36	0.21	0.26	3971
Neutral	0.49	0.62	0.55	5935
Positive	0.23	0.25	0.24	2375
Accuracy			0.41	12281
Macro Avg	0.36	0.36	0.35	12281
Weighted Avg	0.40	0.41	0.40	12281

Table 3. Classification report for Llama 3.2-1B

Class	Precision	Recall	F1-Score	Support
Negative	0.37	0.09	0.14	3972
Neutral	0.48	0.92	0.63	5937
Positive	0.00	0.00	0.00	2375
Accuracy			0.47	12284
Macro Avg	0.28	0.34	0.26	12284
Weighted Avg	0.35	0.47	0.35	12284

Table 4. Classification report for LayerSkip 3.2-1B

LayerSkip models. Each model was prompted to classify sentiment in the Twitter Sentiment Dataset across three categories: negative, neutral, and positive. The classification

outputs were evaluated using standard metrics, including precision, recall, and F1-score.

Although neither model was fine-tuned for the task or dataset, both achieved moderately reasonable performance. The base LLaMA model attained an accuracy of 0.4125, while the LayerSkip model achieved 0.4735. Tables 3 and 4 summarize the evaluation metrics, which demonstrate significant variance across the predicted classes.

Notably, the LayerSkip model failed to predict any positive sentiment samples, resulting in a precision, recall, and F1-score of zero for that class. This also contributed to its disproportionately high recall score for neutral sentiment, as shown in Table 4. Similar tendencies were observed on two additional binary sentiment datasets—Yelp Polarity and SST-2—where the model consistently defaulted to negative sentiment. One plausible explanation is that the layer dropout strategy employed during LayerSkip training may have inadvertently weakened the model’s ability to generalize representations of positive sentiment. Future work could investigate whether this behavior persists in larger LayerSkip variants, such as the 2–7B models, to determine whether this is an isolated anomaly of the 3.2–1B model or a broader limitation of the LayerSkip technique.